Bases de données
SQL

Clés primaires et secondaires
Clés relatives et absolues
Clés externes ou internes

SQL_07
v131a

2026-07-01

Christina.Khnaisser@USherbrooke.ca
Luc.Lavoie@USherbrooke.ca

© 2018-2021, **Μῆτις** (http://info.usherbrooke.ca/llavoie)
CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/)

Plan

- Clés primaires et secondaires
- Clés relatives et absolues
- Clés externes et internes
- Générateurs de clés internes
  - IDENTITY
  - SEQUENCE
  - SERIAL et les autres
- Quelques utilisations remarquables des clés

Clés

Pourquoi ?

- «Pour identifier (et donc distinguer) ce qui doit l'être.»
- Les clés contribuent à intégrer à faible cout d'importantes contraintes d'intégrité.
- Une clé n'en remplace pas une autre, elle s'y ajoute.
  - La raison d'un tel ajout doit donc être explicite et motivée.

Clé (rappel)

- En regard d'une relation, une clé est un ensemble d'attributs qui détermine fonctionnellement tous les autres attributs d'une relation. Ainsi, deux tuples d'une même relation ne peuvent avoir la même (valeur de) clé. Une clé est dite *stricte* si aucun de ses attributs ne peut en être retiré sans qu'elle perde la propriété de clé. La notion de clé s'applique tout aussi bien aux relations qu'aux classes et aux ensembles d'entités. Une clé sera qualifiée de *locale* si ses attributs appartiennent à la même relation que les tuples déterminés, et *référentielle* s'ils appartiennent aux tuples d'une autre relation (dite *référée*). Une clé référentielle correspond donc à une clé locale de la table référée. Une clé stricte locale est parfois appelée *clé candidate*, un calque de *candidate key* en langue états-unienne. Lorsqu'une relation possède plusieurs clés strictes locales, l'une d'entre elles est souvent distinguée sous le nom de clé primaire, les autres étant alors des clés secondaires.

Orthographe

- Le nom *clé* est fréquemment utilisé en apposition, par exemple *un attribut clé*. Au pluriel, conformément à la règle régissant l'apposition, il reste invariable, *des attributs clé*. Il est aussi utilisé pour caractériser une entité qui n'est pas une clé, *une non-clé, des attributs non-clé*, **avec** un trait d'union comme le prescrit la règle régissant la négation des noms. Votre correcteur orthographique n'est pas de cet avis ? Alors, il s'agit vraisemblablement d'un outil utilisant l'IA, mais incapable de conjuguer deux règles de façon priorisée !

Usage.

- En pratique, la détermination des clés s'intéresse essentiellement aux clés strictes, les clés non strictes pouvant alors être aisément déduites. Il est donc fréquent que la dénomination *clé* soit utilisée implicitement pour *clé stricte*. De plus, une clé non qualifiée de référentielle sera considérée locale. Dans le contexte de textes informels, il faut donc souvent traduire *clé* par *clé stricte locale* et *clé référentielle* par *clé stricte référentielle*.

Clés primaires et secondaires

- Définition
- Origines
- Utilité
- Critères de choix
- Représentation sous SQL

Clés primaires et secondaires
Définition et origine

- Définition
  - Parmi les clés locales strictes d'une relation, une d'elle est désignée comme clé primaire; les autres (s'il en est) sont désignées comme clés secondaires.
- Origine de la différenciation
  - Les limites des fichiers séquentiels indexés… en 1966!

 © 2018-2026, MyTuq — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Clés primaires et secondaires
Utilité

- Pertinence du concept
  - Ce concept est inutile du strict point de vue de la théorie relationnelle.
- Toutefois
  - Le choix d'une clé locale stricte de référence (clé primaire) peut contribuer à la clarté et à la modifiabilité du schéma, particulièrement lors de la définition de clés référentielles, ce qui importe du point de vue du génie logiciel.
  - En dernier ressort, le choix d'une clé plutôt qu'une autre peut avoir une incidence sur la performance en fonction des techniques d'indexation utilisées.

Clés primaires et secondaires
Critère de choix

- Flou
  - Nombre d'attributs constitutifs ?
  - Taille totale ?
  - Flexibilité au moment des jointures ?
  - Flexibilité au moment des projections (donc des groupements) ?

- En pratique, le choix dépend tout autant de considérations d'utilisation (évoluant dans le temps) que de considérations de représentation (variant d'un SGBD à l'autre).

Clés primaires
Représentation sous SQL

- Clé primaire
  - PRIMARY KEY
  - les attributs la composant sont automatiquement marqués « not null »
- Clés secondaires
  - UNIQUE
  - les attributs les composant **ne sont pas** automatiquement marqués « not null »
    - ce qui est une erreur, aucun attribut d'une clé ne peut être annulable!

Clés relatives et absolues

- Rappels
- Définitions
- Pourquoi ?
- Quand ?
- Avantages (et inconvénients)
- Coexistence ?

Rappel

- Ci-après, sauf indication contraire, le terme *clé* désigne une clé irréductible (*candidate key*). Au besoin, lorsqu'il sera nécessaire de désigner tout sous-ensemble d'attributs identifiant les tuples d'une relation, nous utiliserons l'expression *clé irréductible ou non* (ou l'équivalent, *clé stricte ou non*).
- Une relation possède toujours une clé (à la limite l'ensemble de ses attributs à défaut de tout autre).

Clé propre
Définition

- Une clé est dite *propre* si et seulement si :
  - aucun sous-ensemble de ses attributs n'est une clé référentielle.
- Une clé propre est dite *absolue* si et seulement si :
  - elle ne comporte qu'un seul attribut ;
- Une clé absolue représente le fait qu'une relation comporte un attribut qui lui est propre et qui identifie la totalité de ses tuples.

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Mytiş — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Clé relative
Définition

- Une clé est dite relative si et seulement si :
  - au moins un sous-ensemble de ses attributs est une clé référentielle ;
- Une clé relative est dite
  - *partielle*, si au moins un attribut lui est propre (c'est-à-dire ne participant à aucune clé référentielle) ;
  - *totale*, sinon.
- Corolaires :
  - toute clé est soit propre, soit relative.
  - une clé relative partielle comporte au moins deux attributs.

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Que se passe-t-il si un attribut de la clé locale de référence fait partie de clés référentielles mais qu'aucunes de ces dernières n'est un sous-ensemble de la clé locale de référence ?

Clés relatives
Pourquoi ? Quand ?

- Les clés relatives sont nécessaires lorsqu'une dépendance fonctionnelle doit être modélisée entre relations.
- Les clés relatives sont donc utilisées lors de la traduction du schéma conceptuel vers le schéma logique.
- En particulier, pour la représentation d'entités faibles en regard des schémas EA.

- Voir module MCD_04.

Coexistence des clés absolues et relatives

- Certains auteurs recommandent préférentiellement les clés absolues comme cible des clés référentielles, voire même d'en créer artificiellement une s'il n'y a pas de clé absolue externe.
- Cette tactique est rarement payante en pratique attendu les algorithmes contemporains d'indexation.
- Au contraire, ceci concourt souvent à augmenter le nombre de jointures dans les requêtes.

Clés internes et externes

- Définition
- Pourquoi ?
- Quand ?
- Comment ?

internes

- **artificielles**
- **subsidiaires**

externes

- **naturelles**

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Clé interne

- Clé absolue n'ayant aucune sémantique associée en regard du domaine d'application.
- En pratique, une clé interne doit donc être maintenue par le SGBD en regard du schéma et de la table ciblée, puisqu'elle ne provient pas de la réalité modélisée.

- Complément
  - Une clé est dite externe, si et seulement si elle n'est pas interne !

 © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Clé interne
Quand doit-on l'utiliser ?

- Pour distinguer ce qui doit l'être en l'absence de clé externe (éviter les amalgames accidentels)
  - *exemple* : les humains, les animaux, les plantes …
- Pour éviter la propagation d'informations sensibles contenues dans les clés externes
  - *exemple* : le NAS (numéro d'assurance sociale)
- Pour éviter que la clé d'une entité retirée soit réutilisée
  - *exemple* : l'unicité et la pérénnité des clés externes sont rarement garanties à long terme

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, MyTuq — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Attention, ne jamais associer une signification à la valeur d'une clé interne, sinon elle n'est plus interne !

Clé interne
Quand peut-on l'utiliser ?

- Pour faciliter que la clé d'une entité retirée soit réutilisée
  - *exemple* : l'unicité et l'immuabilité des clés externes sont rarement garanties à long terme
- Pour pallier les contingences externes relativement à la définition et la modification d'une clé externe
  - *exemple* : l'unicité et la pérénnité des clés externes sont rarement garanties à long terme
- Pour optimiser le temps de calcul
  - *exemple* : clé externe TRÈS volumineuse (enfin, suffisamment volumineuse pour avoir un impact réel sur les ressources).

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Mytiş — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Une clé externe est le plus souvent définie par une une instace externe, tout comme son iinterprétation.
Il peut alors être risqué de faire dépendre le modèle de contingences externes.
Pour pallier ce risque, une clé interne est ajoutée : elle ne peut ni être modifiée, ni perdre sa propriété de clé.
Ceci est notamment utile quant un suivi de l'entité doit pouvoir être maintenu indépendamment du changement de valeur de la clé externe.

Autres cas ?

Clé interne
Quand ne doit-on pas l'utiliser ?

- On ne doit pas remplacer une clé relative par un clé interne. Tout au plus est-il possible de l'ajouter, et encore avec une solide justification.
- En pratique, il y a peu d'avantages concrets à cette ajout. Dans certains cas, il y a même une perte notable de performance et un emcombrement supérieur.
- Pour plus de détails, voir [Sainte-Marie].

Clé interne et code

Un code n'est pas une clé interne !

- Un code est une clé externe dotée sémantique en regard du domaine d'application.
- Le code peut résulter d'une convention connue ou être généré à l'interne comme à l'externe.
  - *exemples* : les symboles atomiques du tableau péridodique, les abréviations, les acronymes.
- Un code a souvent une valeur mnémotechnique afin
  - de servir à fins d'affichage, par souci de compacité dans des tableaux, par exemple;
  - de pouvoir servir dans la programmation des requêtes.
- Un code est donc contingent à des règles externes et ne peut être modifié sans accord (obtenu explicitement ou par une règle).
- Un code n'est pas une clé interne!

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Un code est souvent représenté par une courte séquence alphanumérique.

Clés internes

Exemples de codes qui sont de mauvaises clés internes

- Le matricule de la Régie d'assurance-maladie du Québec RAMQ
  - référence au nom, prénom et date de naissance
- Le numéro d'assurance sociale
  - référence au statut de résidence
- Le code d'identification personnel (CIP) de l'Université de Sherbrooke
  - référence au nom et prénom
- Le matricule étudiant de l'Université de Sherbrooke
  - référence au nom, prénom, date de naissance et date d'admission
- Le matricule employé de l'Université de Sherbrooke
  - référence à la date d'embauche

- Constat
  - En pratique, il n'est de bonne clé interne que celles qui *ne proviennent pas* du domaine d'application. La tentation de permettre d'en déduire des informations, souvent sensibles, semble trop forte.

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, MyTuq — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Note

Si ces codes ne sont pas de bonnes clés internes, cela n'empêche pas qu'ils puissent être (ou pas) de bons moyens d'identification ☺

Clés internes
Comment les générer ?

- En SQL
  - SEQUENCE
- Dans de nombreux dialectes SQL
  - SMALLSERIAL
  - SERIAL
  - BIGSERIAL
  - UID
  - GUID
  - UUID

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Génération de clés internes en SQL

- Caractéristiques souhaitées
- Les solutions
  - IDENTITY
  - SEQUENCE
    - générateurs
    - fonctions
  - SERIAL, SMALLSERIAL, BIGSERIAL
  - UID, GUID, UUID

Génération de clés internes en SQL
Caractéristiques souhaitées

- Artificialité garantie lors de la génération
- Unicité garantie lors de la génération
- Unicité garantie dans un contexte transactionnel
- Automatisation de la génération lors de l'insertion

 © 2018-2026, MyTuς — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Portée

Clés internes en SQL ISO
Solution 1 : Par annotation de type (IDENTITY)

type_identitaire ::=
  type GENERATED [ALWAYS | AS DEFAULT]
  AS IDENTITY *séquence*

séquence ::=
  nom_de_sequence | *générateur*

Notes relatives à PostgreSQL

- Le *type* est limité aux seuls types numériques entiers (SMALLINT, INTEGER, BIGINT).
- La mise à disposition du mécanisme ISO a été graduelle, depuis la version 9.6 jusqu'à la 12.4.

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Le type réfère normalement à un type scalaire textuel ou numérique.
La restriction aux seuls types entiers en PostgreSQL est incommode.

Clés internes en SQL ISO
Solution 2 : Création d'un générateur (SEQUENCE)

création_de_séquence ::=
CREATE [TEMPORARY | TEMP]
SEQUENCE *nom_de_séquence* [AS *type*]
générateur
[OWNED BY { *nom_table.nom_colonne* | NONE }]

générateur ::=
[INCREMENT [BY] *incrément*]
[MINVALUE *valeur_min* | NO MINVALUE]
[MAXVALUE *valeur_max* | NO MAXVALUE]
[START [WITH] *début*]
[CACHE *cache*]
[[NO] CYCLE]

La clause OWNED BY est une extension PostgreSQL

Remarquer qu'un générateur peut se réduire à... rien du tout! Dans ce cas, un générateur implicite est utilisé.

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

type, un type entier – bigint par défaut

...
CACHE nombre de valeurs générées à la fois

TEMPORARY « masquer une séquence durant une session »

OWNED BY une extension PostgreSQL

SELECT * FROM <nom-de-sequence>

Clés internes en SQL ISO
Solution 2 : Utilisation d'un générateur (SEQUENCE)

- Pour obtenir *implicitement* la prochaine valeur, lors d'une insertion, ne pas donner de valeur à l'attribut !
- Pour obtenir *explicitement* la prochaine valeur, utiliser la fonction standard NEXT VALUE FOR.

Notes relatives à PostgreSQL

- Le mécanisme *implicite* est le même.
- Pour le mécanisme *explicite*, voire la diapositive suivante.

Clés internes en SQL ISO
Solution 2 : Utilisation d'un générateur (SEQUENCE)… PostgreSQL

setval (regclass, bigint)	bigint	Définit la valeur courante de la séquence
setval (regclass, bigint, boolean)	bigint	Définit la valeur courante de la séquence et son statut (booléen)
currval (regclass)	bigint	Renvoie la valeur la plus récemment obtenue avec nextval pour la séquence indiquée
lastval ()	bigint	Renvoie la valeur la plus récemment obtenue avec nextval (quelle que soit la séquence)
nextval (regclass)	bigint	Incrémente la séquence et renvoie la nouvelle valeur

Les objets sequence sont de type regclass
voir https://docs.postgresql.fr/15/functions-sequence.html

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

setval (s, v [, b])
Définit la valeur du compteur de l'objet séquence. La forme avec deux paramètres initialise le champ last_value de la séquence à la valeur précisée et initialise le champ is_called à true, signifiant que le prochain nextval avance la séquence avant de renvoyer une valeur. La valeur renvoyée par currval est aussi configuré à la valeur indiquée. Dans la forme à trois paramètres, is_called peut être initialisé à true ou à false. true a le même effet que la forme à deux paramètres. Positionné à false, le prochain nextval retourne exactement la valeur indiquée et l'incrémentation de la séquence commence avec le nextval suivant. De plus, la valeur indiquée par currval n'est pas modifiée dans ce cas.
currval (s)
Renvoie la valeur courantel pour cette séquence dans la session courante. Parce qu'elle renvoie une valeur locale à la session, la réponse est prévisible, que d'autres sessions aient exécuté ou non la fonction nextval après la session en cours.
lastval ()
Renvoie la valeur courante dans la session courante. Cette fonction est identique à currval, sauf qu'au lieu de prendre le nom de la séquence comme

argument, elle récupère la valeur de la dernière séquence utilisée par nextval dans la session en cours. Si nextval n'a pas encore été appelée dans la session en cours, un appel à lastval produit une erreur.

nextval (s)

Avance l'objet séquence à sa prochaine valeur et renvoie cette valeur. Ce fonctionnement est atomique : même si de multiples sessions exécutent nextval concurrentiellement, chacune obtient sans risque une valeur de séquence distincte.

Par exemple

SELECT setval('foo', 42);	*Le nextval suivant retourne 43*
SELECT setval('foo', 42, true);	*Comme ci-dessus*
SELECT setval('foo', 42, false);	*Le nextval suivant retourne 42*

SEQUENCE
Remarque importante

- Pour éviter le blocage de transactions concurrentes qui obtiennent des nombres de la même séquence, une opération *nextval* n'est jamais annulée ; c'est-à-dire qu'une fois la valeur récupérée, elle est considérée utilisée, même si la transaction qui exécute *nextval* avorte par la suite. Cela signifie que les transactions annulées peuvent laisser des « trous » inutilisés dans la séquence des valeurs assignées.

 © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

SEQUENCE
SERIAL - définition

- Le type SERIAL n'est pas un vrai type, mais plutôt un raccourci pour créer des valeurs uniques de type INTEGER :

```
CREATE TABLE nom_de_table (
  nom_de_colonne SERIAL,
  ...
);
```

- est équivalent à écrire :

```
CREATE SEQUENCE nom_de_table_nom_de_colonne_seq;
CREATE TABLE nom_de_table (
  nom_de_colonne INTEGER
    GENERATED AS DEFAULT AS IDENTITY nom_de_table_nom_de_colonne_seq,
  ...
);
ALTER SEQUENCE nom_de_table_nom_de_colonne_seq
  OWNED BY nom_de_table.nom_de_colonne;
```

- Un attribut de type entier est ainsi créé dont la valeur par défaut est obtenue par un générateur de séquence propre à l'attribut.
- Enfin, la séquence est marquée OWNED BY (possédée par) l'attribut pour qu'elle soit automatiquement supprimée si l'attribut ou la table est supprimé.

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

SEQUENCE
SERIAL – un exemple en SQL ISO

Ceci

```
CREATE TABLE T (
    C SERIAL,
    ...
);
```

est équivalent à écrire :

```
CREATE SEQUENCE T_C_seq;
CREATE TABLE T (
    C INTEGER
        GENERATED AS DEFAULT AS IDENTITY T_C_seq,
    ...
);
ALTER SEQUENCE T_C_seq
    OWNED BY T.C;
```


2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

SEQUENCE
SERIAL – un exemple en PostgreSQL

Ceci

```
CREATE TABLE T (
    C SERIAL,
    ...
);
```

est équivalent à écrire :

```
CREATE SEQUENCE T_C_seq;
CREATE TABLE T (
    C INTEGER NOT NULL
        DEFAULT nextval('T_C_seq'),
    ...
);
ALTER SEQUENCE T_C_seq
    OWNED BY T.C;
```


2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

SEQUENCE
SERIAL – notes

- SMALLSERIAL, SERIAL et BIGSERIAL sont mis en oeuvre en utilisant des séquences correspondant respectivement à SMALLINT, INTEGER et BIGINT; ils en ont donc les caractéristiques.
- Dans la plupart des cas, une contrainte UNIQUE ou PRIMARY KEY est ajoutée pour interdire que des doublons soient créés par accident, mais ce n'est pas automatique.
- Pour affecter la valeur suivante de la séquence à un attribut, il faut préciser que la valeur par défaut de l'attribut doit être utilisée. Cela peut notamment se faire en excluant cet attribut de la liste des attributs de la commande INSERT.

2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

8.1.4. Types seriés

Les types de données smallserial, serial et bigserial ne sont pas de vrais types, mais plutôt un raccourci de notation pour créer des colonnes d'identifiants uniques (similaires à la propriété AUTO_INCREMENT utilisée par d'autres SGBD). Dans la version actuelle, indiquer :

CREATE TABLE *nom_de_table* (
  nom_de_colonne SERIAL
);

est équivalent à écrire :

CREATE SEQUENCE *nom_de_table_nom_de_colonne*_seq;
CREATE TABLE *nom_de_table* (
  nom_de_colonne integer NOT NULL DEFAULT
nextval('*nom_de_table_nom_de_colonne*_seq') NOT NULL
);
ALTER SEQUENCE *nom_de_table_nom_de_colonne*_seq OWNED BY
nom_de_table.nom_de_colonne;

Une colonne d'entiers a ainsi été créée dont la valeur par défaut est assignée par un générateur de séquence. Une contrainte NOT NULL est ajoutée pour s'assurer qu'une valeur NULL ne puisse pas être insérée. (Dans la plupart des cas, une contrainte UNIQUE ou PRIMARY KEY peut être ajoutée pour interdire que des doublons soient créés par accident, mais ce n'est pas automatique.) Enfin, la séquence est marquée « owned by » (possédée par) la colonne pour qu'elle soit supprimée si la colonne ou la table est supprimée.

Note

Comme smallserial, serial et bigserial sont implémentés en utilisant des séquences, il peut y avoir des trous dans la séquence de valeurs qui apparait dans la colonne, même si aucune ligne n'est jamais supprimée. Une valeur allouée à partir de la séquence est toujours utilisée même si la ligne contenant cette valeur n'est pas insérée avec succès dans la colonne de la table. Cela peut survenir si la transaction d'insertion est annulée. Voir nextval() dans Section 9.16, « Fonctions de manipulation de séquences » pour plus de détails.

Pour insérer la valeur suivante de la séquence dans la colonne serial, il faut préciser que la valeur par défaut de la colonne doit être utilisée. Cela peut se faire de deux façons : soit en excluant cette colonne de la liste des colonnes de la commande **INSERT** soit en utilisant le mot clé DEFAULT.

Les types serial et serial4 sont identiques : ils créent tous les deux des colonnes integer. Les types bigserial et serial8 fonctionnent de la même façon mais créent des colonnes bigint. bigserial doit être utilisé si plus de 2^{31} identifiants sont prévus sur la durée de vie de la table. Les noms de type smallserial et serial2 fonctionnent de la même façon, sauf qu'ils créent une colonne de type smallint.

La séquence créée pour une colonne serial est automatiquement supprimée quand la colonne correspondante est supprimée. La séquence peut être détruite sans supprimer la colonne, mais la valeur par défaut de la colonne est alors également supprimée.

SEQUENCE
SERIAL – un exemple en PostgreSQL

Ceci est le plus souvent préférable

```
CREATE TABLE T (
    C SERIAL,
    ...
    PRIMARY KEY (C)
);
```

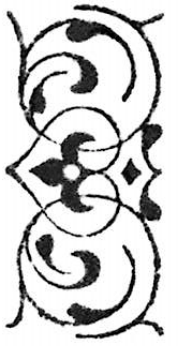
Ce qui est équivalent à :

```
CREATE SEQUENCE T_C_seq;
CREATE TABLE T (
    C INTEGER NOT NULL
        DEFAULT nextval('T_C_seq'),
    ...
    PRIMARY KEY (C)
);
ALTER SEQUENCE T_C_seq
    OWNED BY T.C;
```


2026-07-01 MCED_SQL_07 — Clés (v131a) © 2018-2026, Μῆτις — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

SEQUENCE
Les UID, GUID et autres UUID

- Hors norme SQL.
- Spécifiés par l'ISO, la CEI et l'IETF.
- Bien que le type UUID soit parfois disponible (commen PostgreSQL), les fonctions dont plus souvent disponibles par l'entremise de « packages » (ou «extensions»)
  - https://docs.postgresql.fr/17/datatype-uuid.html
  - https://docs.postgresql.fr/17/functions-uuid.html
  - https://docs.postgresql.fr/17/uuid-ossp.html

Références

- Loney, Kevin ;
  Oracle Database 11g: The Complete Reference.
  Oracle Press/McGraw-Hill/Osborne, 2008.
  ISBN 978-0071598750.
- Date, Chris J. ;
  SQL and Relational Theory: How to Write Accurate SQL Code.
  2nd edition, O'Reilly, 2012.
  ISBN 978-1-449-31640-2.
- Le site d'Oracle (en anglais)
  - http://docs.oracle.com/cd/E11882_01/index.htm
- Le site de PostgreSQL (en français)
  - http://docs.postgresqlfr.org
  - https://docs.postgresql.fr/17/uuid-ossp.html